CISS245: Advanced Programming
Quiz q6401

Name: eagraham1@cougars.ccis.edu Score:

Q1. In the Wumpus world, suppose you have several Wumpuses. When a hunter kills a Wumpus, say `joe_wumpus`, the hunter earns `joe_wumpus.points_`. You think it's reasonable for `joe_wumpus.points_` to be 1000. When the hunter kills any Wumpus, the number of points awarded is the same. Of course each Wumpus has a row and a column for its position in the Wumpus World. Each Wumpus also has a direction (N, S, E, W) which is the direction that it is facing, i.e., if the Wumpus is facing N, then when it moves it will move in the north direction if it does not bump against a wall.

Correct the following header file to optimize on memory usage when you have a huge number of Wumpuses.
Answer:

```
// file: Wumpus.h
class Wumpus
{
public:
    Wumpus()
    : is_alive_(true), row_(0), col_(0), direction_('N')
    {}

    int get_points() const;
    // other methods not shown
private:
    bool is_alive_;
    int row_;
    int col_;
    char direction_;
    static const int points_ = 1000;
};
```

Q2. Refer to the question above. Complete the following by implementing `Wumpus::Wumpus()`, `Wumpus::get_points()`, and whatever is necessary. You need not implement methods not shown in the previous question. (There is something else besides the above two functions.)
Answer:

```
// file: Wumpus.cpp
#include "Wumpus.h"
```

```
int get_points() const
{
  return points;
}
```

Q3. The `DollarAmount` class behaves like this:

```
DollarAmount amt0(1);
DollarAmount amt1(1234);
std::cout << amt0 << '\n'; // prints "$0.01" (without double-quotes)
std::cout << amt1 << '\n'; // prints "$12.34" (without double-quotes)
DollarAmount::set_format(1);
std::cout << amt0 << '\n'; // prints "1c" (without double-quotes)
std::cout << amt1 << '\n'; // prints "1234c" (without double-quotes)
DollarAmount::set_format(0);
std::cout << amt0 << '\n'; // prints "$0.01" (without double-quotes)
std::cout << amt1 << '\n'; // prints "$12.34" (without double-quotes)
```

Complete and correct `DollarAmount.h` so that the above code fragment works. Do not define the bodies of the methods (do that in the next question).
Answer:

```
// file: DollarAmount.h

#ifndef DOLLARAMOUNT_H
#define DOLLARAMOUNT_H

#include <iostream>

class DollarAmount
{
public:
DollarAmount(int cents);
static void set_format(int format);
private:
    int cents_;
    int format_;
};

std::ostream& operator<<(std::ostream& cout, const DollarAmount& amt);
```

Q4. Refer to the previous question. Complete `DollarAmount.cpp` so that the code fragment in the previous question works.
Answer:

```
// file: DollarAmount.cpp

```

```
#include <iostream>
#include "DollarAmount.h"
DollarAmount::DollarAmount(int cents)
: cents_(cents)
{}
void DollarAmount::set_format(int format)
{
  format_ = f;
}

std::ostream& operator<<(std::ostream& cout, const DollarAmount)
{
  if(DollarAmount::format__ == 0)
  {
    int dollars = amt.cents_ / 100;
    int cents = amt.cent_ % 100;
    cout << '$' << dollars << std::setw(2) << std::setfill('0') << cents;
  }
  else
  {
    cout << amt.cents_ << 'c';
  }

  return cout;
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.